

Thèse présentée pour l'obtention du grade de  
**DOCTEUR de SORBONNE UNIVERSITÉ**

Spécialité  
**Ingénierie / Systèmes Informatiques**

École doctorale  
**Informatique, Télécommunication et Électronique Paris (ED130)**

**Mécanisme et politique dans le noyau : ordonnancement et  
gestion mémoire**

---

Prénom Nom

Soutenue publiquement le : *1 janvier 2030*

Devant un jury composé de :

**Prénom Nom**, Poste, Université  
**Prénom Nom**, Poste, Université  
**Prénom Nom**, Poste, Université  
**Prénom Nom**, Poste, Université  
**Prénom Nom**, Poste, Entreprise  
**Prénom Nom**, Poste, Université

*Rapporteur*  
*Rapporteur*  
*Examineur*  
*Examinatrice*  
*Invité*  
*Directeur de thèse*

*À qui de droit*



**Copyright:**

Except where otherwise noted, this work is licensed under  
<https://creativecommons.org/licenses/by-nc-nd/4.0/>

# Abstract

Modern kernels are asked to do two things at once: isolate workloads from one another, and get out of their way. This placeholder abstract stands in for the real one. It is long enough to show how the page sets, and short enough to fit on a single page.

The first contribution looks at scheduling. We measure how the CFS behaves when the machine is oversubscribed, and show where its load-balancing heuristics lose work-conserving behaviour on NUMA hardware.

The second contribution is a memory-allocation path built on eBPF, which lets policy be changed without rebuilding the kernel. On our benchmarks it matches the in-tree allocator at low load and improves tail latency under pressure.

**Keywords:** operating systems, kernel, scheduling, memory management, eBPF, NUMA



# Résumé

Les noyaux modernes doivent faire deux choses à la fois : isoler les charges de travail les unes des autres, et ne pas leur faire obstacle. Ce résumé est un texte de remplissage. Il est assez long pour montrer le rendu de la page, et assez court pour tenir sur une seule.

La première contribution porte sur l’ordonnancement. Nous mesurons le comportement du CFS lorsque la machine est sur-souscrite, et montrons où ses heuristiques d’équilibrage perdent la propriété de conservation du travail sur une architecture NUMA.

La seconde contribution est un chemin d’allocation mémoire fondé sur eBPF, qui permet de changer de politique sans recompiler le noyau. Sur nos tests, il égale l’allocateur intégré à faible charge et améliore la latence de queue sous pression.

**Mots-clés :** systèmes d’exploitation, noyau, ordonnancement, gestion mémoire, eBPF, NUMA



# Table des matières

|           |                                             |           |
|-----------|---------------------------------------------|-----------|
| <b>1</b>  | <b>Introduction</b>                         | <b>1</b>  |
| 1.1       | Contexte . . . . .                          | 1         |
| 1.2       | Contributions . . . . .                     | 1         |
| 1.3       | Organisation du manuscrit . . . . .         | 2         |
| <b>I</b>  | <b>Fondations</b>                           | <b>5</b>  |
| <b>2</b>  | <b>Architecture d'un noyau monolithique</b> | <b>6</b>  |
| 2.1       | Vue d'ensemble . . . . .                    | 6         |
| 2.1.1     | Chemin d'un paquet . . . . .                | 7         |
| 2.1.2     | Descripteurs d'anneau . . . . .             | 8         |
| 2.1.3     | Formats comparés . . . . .                  | 8         |
| 2.2       | Le chemin d'ordonnancement . . . . .        | 8         |
| 2.3       | Conclusion . . . . .                        | 9         |
| <b>II</b> | <b>Contributions</b>                        | <b>11</b> |
| <b>3</b>  | <b>Conception</b>                           | <b>12</b> |
| 3.1       | Principe . . . . .                          | 12        |
| 3.2       | Allocation . . . . .                        | 12        |
| 3.3       | Ordonnancement . . . . .                    | 14        |
| 3.4       | Conclusion . . . . .                        | 14        |
| <b>4</b>  | <b>Évaluation expérimentale</b>             | <b>15</b> |
| 4.1       | Dispositif . . . . .                        | 15        |
| 4.2       | Débit . . . . .                             | 16        |
| 4.3       | Latence . . . . .                           | 17        |
| 4.4       | Comparaison aux travaux existants . . . . . | 17        |
| 4.5       | Synthèse . . . . .                          | 18        |
| <b>5</b>  | <b>Conclusion</b>                           | <b>19</b> |
| 5.1       | Bilan . . . . .                             | 19        |
| 5.2       | Perspectives . . . . .                      | 19        |
|           | <b>Bibliographie</b>                        | <b>21</b> |

|                                         |           |
|-----------------------------------------|-----------|
| <b>Glossaire</b>                        | <b>23</b> |
| <b>Table des figures</b>                | <b>25</b> |
| <b>Liste des tableaux</b>               | <b>27</b> |
| <b>Liste des listings</b>               | <b>29</b> |
| <b>A Détails d'implémentation</b>       | <b>31</b> |
| A.1 Paramètres de compilation . . . . . | 31        |
| A.2 Table des symboles . . . . .        | 31        |

# Introduction

Un *Operating System* (OS) a deux rôles difficiles à concilier : isoler les applications les unes des autres, et leur laisser le matériel sous la main. Ce chapitre pose le problème et annonce le plan. Tout le texte de ce squelette est du remplissage : remplacez-le, gardez la structure.

## 1.1 Contexte

Depuis [RT74], l'interface entre application et noyau a peu changé, alors que le matériel a changé du tout au tout : plusieurs dizaines de cœurs, mémoire *Non-Uniform Memory Access* (NUMA), cartes réseau capables de saturer un *Central Processing Unit* (CPU) à elles seules. Les architectures alternatives explorées par [EKO95] et [BBD09] partent toutes du même constat.

**Le coût de l'abstraction** Chaque couche traversée coûte des cycles. Les travaux sur les plans de données en espace utilisateur [Bel+14; Pet+14] montrent que l'essentiel du coût n'est pas le calcul utile mais le chemin qui y mène.

**Le coût du partage** À l'inverse, retirer le noyau du chemin critique rend le partage plus difficile. La règle de commutativité [CKZ13] donne un critère pour savoir quand une interface peut passer à l'échelle.

## 1.2 Contributions

Ce manuscrit contient deux contributions, une par partie.

La partie I rassemble les éléments nécessaires : modèle d'exécution, hiérarchie mémoire, et les outils de mesure utilisés partout ensuite. La partie II présente l'ordonnanceur et le chemin d'allocation, puis les évalue.

## 1.3 Organisation du manuscrit

Le chapitre 2 décrit l'architecture d'un noyau monolithique moderne. Le chapitre 3 présente la conception. Le chapitre 4 en mesure les performances. Le chapitre 5 conclut.





# Partie I

---

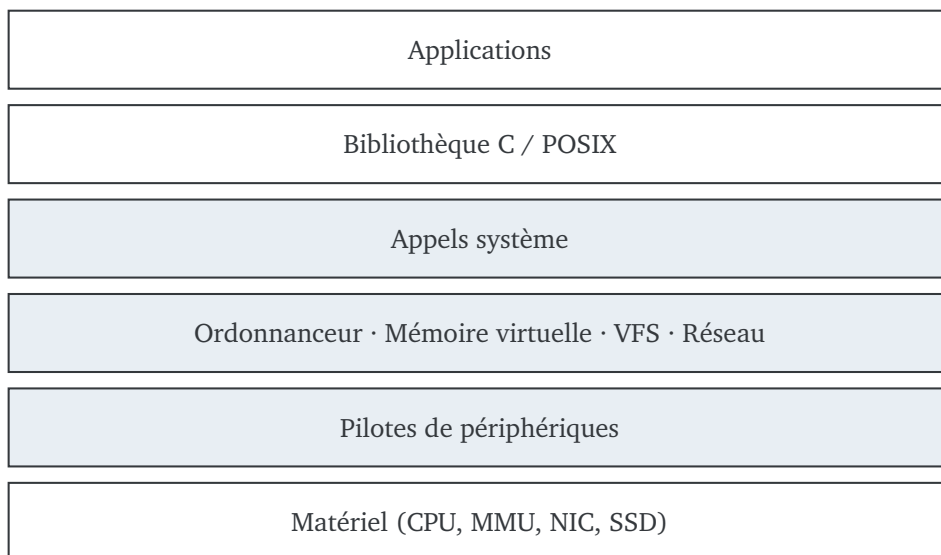
Fondations

# Architecture d'un noyau monolithique

Ce chapitre décrit les sous-systèmes que les contributions viennent modifier. Il sert aussi de démonstration : figure simple, figure large, grille de sous-figures, schéma de paquet annoté, tableau et listing y sont tous utilisés.

## 2.1 Vue d'ensemble

La figure 2.1 donne la vue classique en couches. Une application n'atteint le matériel qu'en traversant l'interface d'appels système, puis le sous-système concerné. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At.



**Fig. 2.1:** Couches traversées par une application, de l'espace utilisateur au matériel.

| Sous-système                     | Unité        | Difficulté principale                                                                   |
|----------------------------------|--------------|-----------------------------------------------------------------------------------------|
| Ordonnanceur                     | tâche        | équité sous sur-souscription                                                            |
| Mémoire virtuelle                | page         | localité NUMA                                                                           |
| <i>Virtual File System</i> (VFS) | inode        | cohérence du cache                                                                      |
| Réseau                           | paquet       | coût par interruption                                                                   |
| Pilotes                          | périphérique | <i>Direct Memory Access</i> (DMA) et <i>Input-Output Memory Management Unit</i> (IOMMU) |

**Tab. 2.1:** Sous-systèmes du noyau et unité de partage.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleam animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificari non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos.

Le tableau 2.1 résume les sous-systèmes et l'unité qu'ils partagent.

### 2.1.1 Chemin d'un paquet

Le trajet d'un paquet entrant est donné figure 2.2 : la carte *Network Interface Card* (NIC) lève une *Interrupt Request* (IRQ), le traitement est différé en *softirq*, puis remis au socket. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleam animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem.



**Fig. 2.2:** Chemin d'un paquet entrant, de la carte réseau à l'application.

### 2.1.2 Descripteurs d’anneau

La figure 2.3 annote le descripteur avec des accolades de regroupement et des notes latérales : les deux premiers mots forment l’en-tête lu par le pilote, les deux suivants l’adresse et la longueur du tampon.

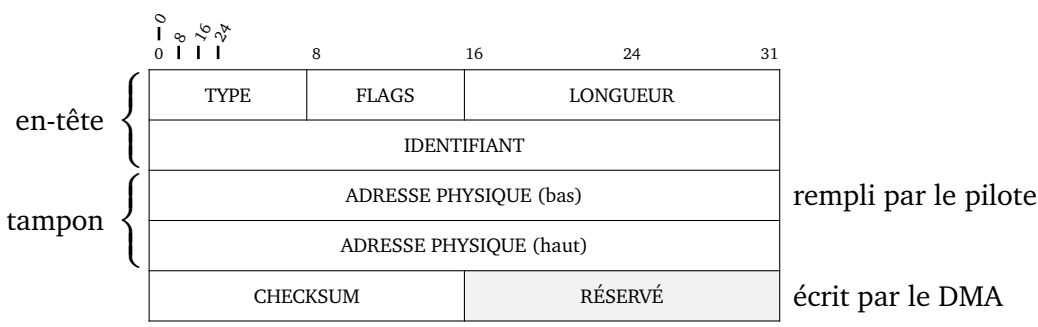


Fig. 2.3: Descripteur d’anneau de réception, avec ses deux groupes de champs.

### 2.1.3 Formats comparés

La figure 2.4 place trois formats côte à côte : les deux premiers, plus courts, sont empilés à gauche ; le troisième occupe la colonne de droite.

## 2.2 Le chemin d’ordonnancement

Le *Completely Fair Scheduler* (CFS) maintient un arbre rouge-noir ordonné par temps d’exécution virtuel [LLF16]. Le listing 2.1 en donne la forme générale. La documentation du noyau [Com26a] décrit les points d’extension disponibles.

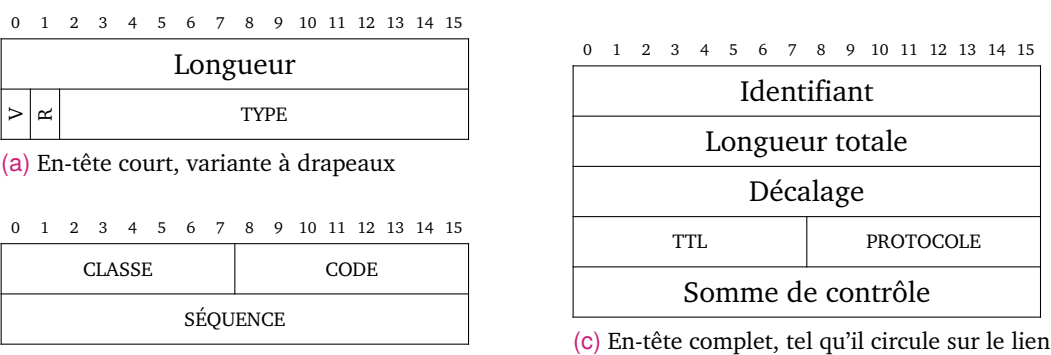


Fig. 2.4: Trois formats d’en-tête, alignés sur la même grille de 16 bits.

```

1 static void enqueue_task_fair(struct rq *rq, struct task_struct *p, int
  flags)
2 {
3     struct cfs_rq *cfs_rq;
4     struct sched_entity *se = &p->se;
5
6     for_each_sched_entity(se) {
7         cfs_rq = cfs_rq_of(se);
8         enqueue_entity(cfs_rq, se, flags);
9         cfs_rq->h_nr_running++;
10    }
11    add_nr_running(rq, 1);
12 }

```

**List. 2.1:** Insertion d'une tâche dans la file d'exécution.

Les verrous protégeant cette structure sont pris par cœur ; la lecture sans verrou repose sur *Read-Copy-Update* (RCU) [MW08]. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

## 2.3 Conclusion

Ces trois chemins — ordonnancement, allocation, réception — se partagent les mêmes structures. La partie suivante s'y attaque.



# Partie II

---

Contributions

# Conception

Ce chapitre décrit les deux mécanismes proposés. Il montre au passage les sous-figures en colonne, les figures larges et le placement flottant.

## 3.1 Principe

L'idée tient en une phrase : déplacer la politique hors du noyau, garder le mécanisme dedans. C'est la séparation défendue par [EKO95], appliquée ici à l'allocation mémoire plutôt qu'au matériel. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit.

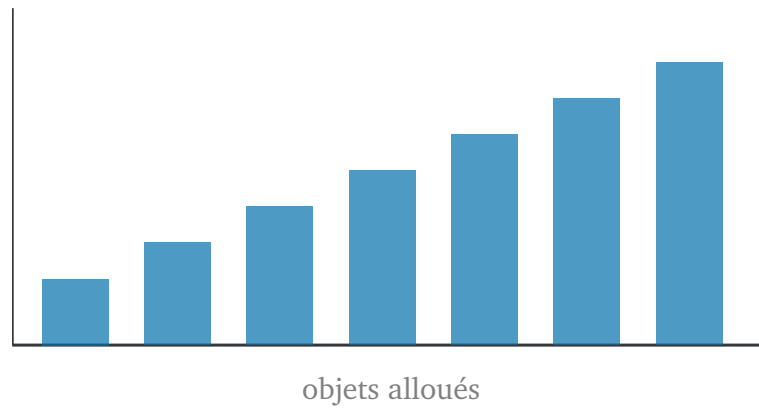
**Mécanisme** Le noyau conserve la propriété des pages et l'invariant d'isolation. Rien de ce qui suit ne peut le violer, puisque le vérificateur *extended Berkeley Packet Filter* (eBPF) rejette tout programme dont la terminaison n'est pas prouvée [CS19] [Com26b].

**Politique** Le choix de la liste, de l'ordre et du seuil de récupération est délégué à un programme chargé à l'exécution. Le changer ne demande pas de recompilation.

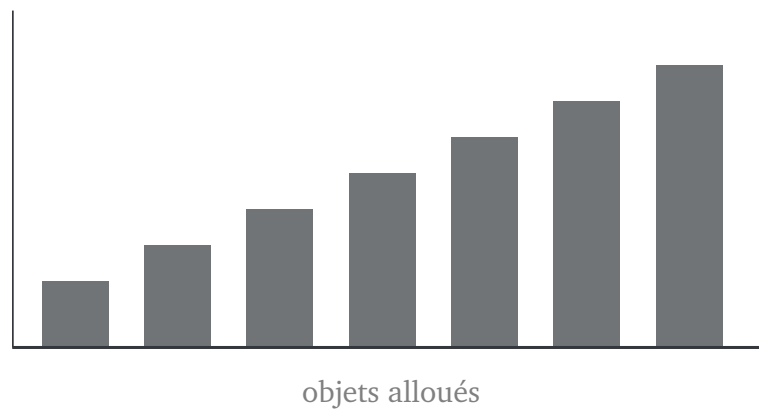
## 3.2 Allocation

La figure 3.1 empile trois stratégies, une par ligne, pour que les axes se comparent verticalement. La sous-figure 3.1a montre l'allocateur par classes [Bon94] ; 3.1b, l'allocateur par binômes ; 3.1c, l'hybride proposé.

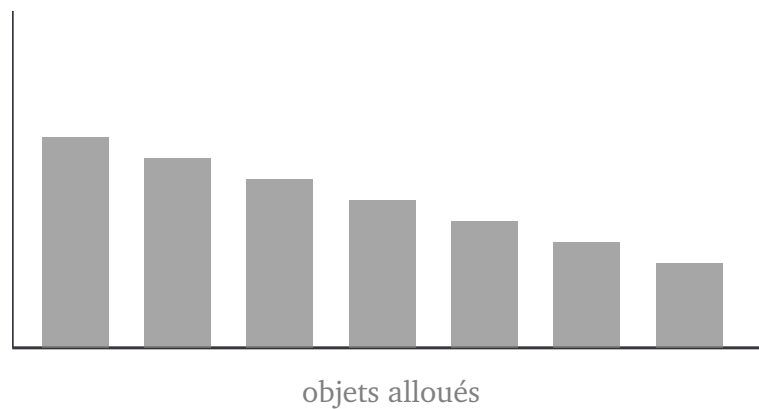
Le comportement diffère surtout en queue de distribution, ce que le chapitre 4 quantifie. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod



(a) Par classes (*Slab Unqueued Allocator (SLUB)*)



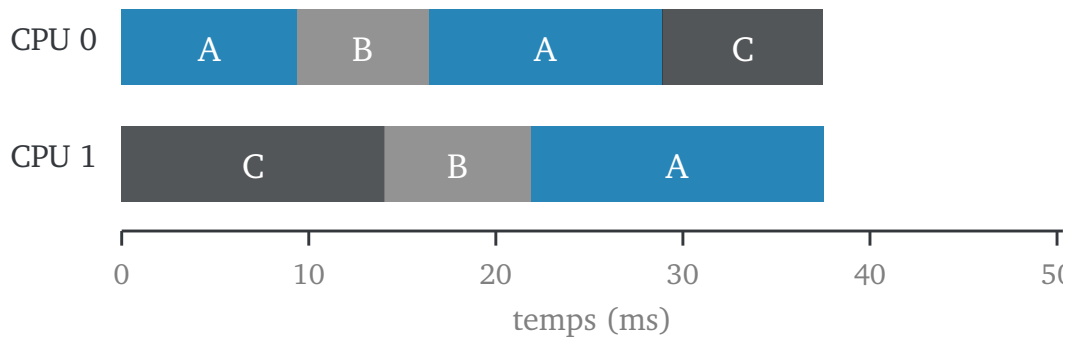
(b) Par binômes



(c) Hybride, politique en eBPF

**Fig. 3.1:** Occupation mémoire des trois allocateurs, à charge identique.

tempor incidunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleam animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificari non possit. At.



**Fig. 3.2:** Trace d'ordonnancement sur deux cœurs. Chaque bloc est un quantum.

### 3.3 Ordonnancement

La figure 3.2 montre deux cœurs sur une fenêtre de 50 ms. La tâche *A* est préemptée deux fois ; la tâche *C* migre de CPU 1 vers CPU 0.

La migration visible au milieu de la trace est le cas que [LLF16] décrit comme une perte de conservation du travail ; [LWN25] en donne une lecture plus récente. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliquam quaerat voluptatem. Ut enim aequaleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem.

### 3.4 Conclusion

Les deux mécanismes partagent la même structure : un point d'attache dans le noyau, une politique remplaçable. Reste à les mesurer.

## Évaluation expérimentale

Ce chapitre mesure les deux mécanismes du chapitre 3. Il montre trois styles de tableau : cellules fusionnées, lignes teintées, et matrice de critères à en-têtes pivotés.

### 4.1 Dispositif

Les mesures utilisent deux machines, décrites tableau 4.1. Les charges sont générées avec les outils de [Gre24], les machines virtuelles par [Pro26]. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem.

| Machine | Matériel |         |       | Rôle             | Noyau                                                     |
|---------|----------|---------|-------|------------------|-----------------------------------------------------------|
|         | Cœurs    | Mémoire | Nœuds |                  |                                                           |
| A       | 64       | 256 Gio | 2     | charge<br>mesure | Linux 6.9, <i>defconfig</i><br>Linux 6.9, eBPF activé     |
| B       | 16       | 64 Gio  | 2     | charge<br>témoin | Linux 6.9, <i>defconfig</i><br>Linux 6.1 LTS, non modifié |

**Tab. 4.1:** Machines utilisées. Les deux sont bi-socket, donc NUMA.

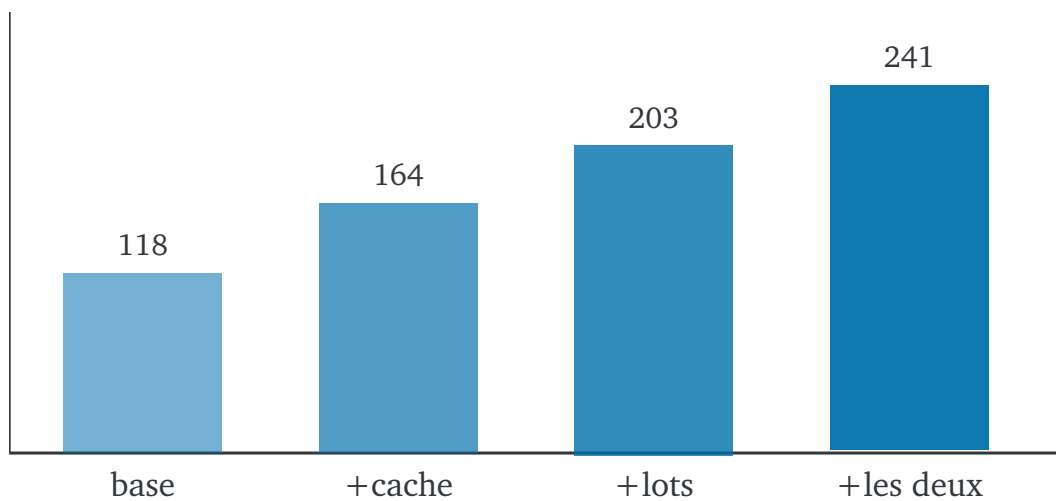
Chaque machine exécute les trois allocateurs, sous trois politiques et sept tailles d'objet. Le tableau 4.2 récapitule ce plan d'expérience ; il est tracé en grille, avec filets verticaux, plutôt qu'en style *booktabs*.

| Allocateur         | Politique | Tailles d'objet (octets)              |
|--------------------|-----------|---------------------------------------|
| Par classes (SLUB) | LIFO      | 16 / 32 / 64 / 128 / 256 / 512 / 1024 |
|                    | FIFO      |                                       |
|                    | LRU       |                                       |
| Par binômes        | LIFO      | 16 / 32 / 64 / 128 / 256 / 512 / 1024 |
|                    | FIFO      |                                       |
|                    | LRU       |                                       |
| Hybride (eBPF)     | LIFO      | 16 / 32 / 64 / 128 / 256 / 512 / 1024 |
|                    | FIFO      |                                       |
|                    | LRU       |                                       |

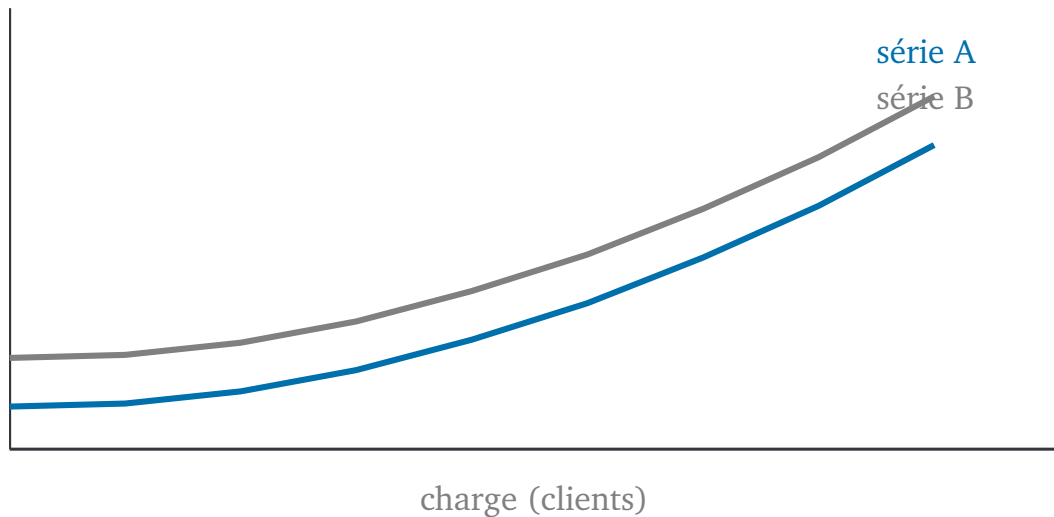
**Tab. 4.2:** Plan d'expérience : trois allocateurs × trois politiques × sept tailles.

## 4.2 Débit

La figure 4.1 donne le débit des quatre variantes. Le gain se lit d'abord sur le cache, puis sur le traitement par lots ; les deux se composent. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.



**Fig. 4.1:** Débit soutenu, en milliers d'opérations par seconde.



**Fig. 4.2:** Latence en fonction de la charge, pour deux configurations.

## 4.3 Latence

La série A de la figure 4.2 reste sous le seuil visé jusqu'à la saturation ; la série B décroche plus tôt.

Le tableau 4.3 reprend ces chiffres. L'écart à la référence est coloré : vert pour un gain, rouge pour une régression. La meilleure valeur de chaque colonne est en gras.

| Variante        | Débit      | $\Delta$ débit | p50                        | p99                         |
|-----------------|------------|----------------|----------------------------|-----------------------------|
| référence       | 118        | —              | 42 $\mu$ s                 | 310 $\mu$ s                 |
| cache seul      | 164        | +39 %          | 38 $\mu$ s                 | 240 $\mu$ s                 |
| lots seuls      | 203        | +72 %          | 36 $\mu$ s                 | 180 $\mu$ s                 |
| cache et lots   | <b>241</b> | <b>+104 %</b>  | <b>31<math>\mu</math>s</b> | <b>120<math>\mu</math>s</b> |
| politique naïve | 96         | -19 %          | 55 $\mu$ s                 | 410 $\mu$ s                 |

**Tab. 4.3:** Débit (kops/s) et latences par variante, écart relatif à la référence.

## 4.4 Comparaison aux travaux existants

Le tableau 4.4 compare notre approche aux systèmes de l'état de l'art sur cinq critères. Les en-têtes sont pivotés pour tenir en largeur.

| Système                   | Isolation | Compatibilité | Transparence | Efficacité | Reconfigurable |
|---------------------------|-----------|---------------|--------------|------------|----------------|
| Noyau monolithique [RT74] | ✓         | ✓             | ✓            | ~          | ✗              |
| Exokernel [EKO95]         | ~         | ✗             | ✗            | ✓          | ✓              |
| Multikernel [BBD09]       | ✓         | ✗             | ✗            | ✓          | ~              |
| Plan de données [Bel+14]  | ~         | ✗             | ✗            | ✓          | ✗              |
| Virtualisation [Bar+03]   | ✓         | ✓             | ~            | ~          | ✗              |
| Cette thèse               | ✓         | ✓             | ✓            | ✓          | ✓              |

**Tab. 4.4:** Respect des cinq critères, par système. ✓ satisfait, ~ partiel, ✗ non satisfait.

## 4.5 Synthèse

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificari non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos.

# Conclusion

## 5.1 Bilan

Ce squelette contient un exemple de chaque élément dont une thèse a besoin : parties, chapitres, figures, sous-figures, tableaux, listings, schéma de paquet, glossaire, bibliographie et annexe. Remplacez le texte, gardez la charpente.

## 5.2 Perspectives

**Court terme** Étendre la mesure aux machines à plus de deux sockets, où la distance NUMA n'est plus binaire.

**Long terme** Appliquer la même séparation mécanisme/politique au VFS, dont le cache pose les mêmes questions de localité.



# Bibliographie

- [Bar+03] Paul Barham, Boris Dragovic, Keir Fraser, et Steven Hand. Xen and the Art of Virtualization. ACM, 2003 (cf. p. 18).
- [BBD09] Andrew Baumann, Paul Barham, et Pierre-Evariste Dagand. The Multikernel: A New OS Architecture for Scalable Multicore Systems. ACM, 2009 (cf. p. 1, 18).
- [Bel+14] Adam Belay, George Prekas, Ana Klimovic, et Samuel Grossman. IX: A Protected Dataplane Operating System for High Throughput and Low Latency. USENIX Association, 2014 (cf. p. 1, 18).
- [Bon94] Jeff Bonwick. The Slab Allocator: An Object-Caching Kernel Memory Allocator. USENIX Association, 1994 (cf. p. 12).
- [CKZ13] Austin T. Clements, M. Frans Kaashoek, et Nickolai Zeldovich. The Scalable Commutativity Rule: Designing Scalable Software for Multicore Processors. ACM, 2013 (cf. p. 1).
- [CS19] Jonathan Corbet et Alexei Starovoitov. BPF: The Universal In-Kernel Virtual Machine. 2019 (cf. p. 12).
- [EKO95] Dawson R. Engler, M. Frans Kaashoek, et James O'Toole. Exokernel: An Operating System Architecture for Application-Level Resource Management. ACM, 1995 (cf. p. 1, 12, 18).
- [LLF16] Jean-Pierre Lozi, Baptiste Lepers, et Justin Funston. The Linux Scheduler: A Decade of Wasted Cores. ACM, 2016 (cf. p. 8, 14).
- [MW08] Paul E. McKenney et Jonathan Walpole. Introducing Technology into the Linux Kernel: A Case Study. ACM, 2008 (cf. p. 9).
- [Pet+14] Simon Peter, Jialin Li, Irene Zhang, et Dan R. K. Ports. Arrakis: The Operating System is the Control Plane. USENIX Association, 2014 (cf. p. 1).
- [RT74] Dennis M. Ritchie et Ken Thompson. The UNIX Time-Sharing System. ACM, 1974 (cf. p. 1, 18).

## Webpages

- [@Com26b] The Linux Kernel Community. BPF Documentation. 2026. URL: <https://docs.kernel.org/bpf/> (cf. p. 12).
- [@Com26a] The Linux Kernel Community. The Linux Kernel Documentation. 2026. URL: <https://www.kernel.org/doc/html/latest/> (cf. p. 8).

- [@Gre24] Brendan Gregg. Linux Performance Tools. 2024. URL: <https://github.com/brendangregg/perf-tools> (cf. p. 15).
- [@LWN25] LWN.net. Completing the EEVDF Scheduler. 2025. URL: <https://lwn.net/Articles/969062/> (cf. p. 14).
- [@Pro26] The QEMU Project. QEMU System Emulation User's Guide. 2026. URL: <https://www.qemu.org/docs/master/system/> (cf. p. 15).

# Glossaire

|              |                                            |
|--------------|--------------------------------------------|
| <b>ABI</b>   | <i>Application Binary Interface</i>        |
| <b>API</b>   | <i>Application Programming Interface</i>   |
| <b>CFS</b>   | <i>Completely Fair Scheduler</i>           |
| <b>CPU</b>   | <i>Central Processing Unit</i>             |
| <b>DMA</b>   | <i>Direct Memory Access</i>                |
| <b>eBPF</b>  | <i>extended Berkeley Packet Filter</i>     |
| <b>HPC</b>   | <i>High-Performance Computing</i>          |
| <b>IOMMU</b> | <i>Input-Output Memory Management Unit</i> |
| <b>IPC</b>   | <i>Inter-Process Communication</i>         |
| <b>IRQ</b>   | <i>Interrupt Request</i>                   |
| <b>LRU</b>   | <i>Least Recently Used</i>                 |
| <b>MMU</b>   | <i>Memory Management Unit</i>              |
| <b>NIC</b>   | <i>Network Interface Card</i>              |
| <b>NUMA</b>  | <i>Non-Uniform Memory Access</i>           |
| <b>OS</b>    | <i>Operating System</i>                    |
| <b>PID</b>   | <i>Process Identifier</i>                  |
| <b>POSIX</b> | <i>Portable Operating System Interface</i> |
| <b>QEMU</b>  | <i>Quick Emulator</i>                      |
| <b>RCU</b>   | <i>Read-Copy-Update</i>                    |
| <b>SLUB</b>  | <i>Slab Unqueued Allocator</i>             |
| <b>SMP</b>   | <i>Symmetric Multiprocessing</i>           |
| <b>SSD</b>   | <i>Solid-State Drive</i>                   |
| <b>TLB</b>   | <i>Translation Lookaside Buffer</i>        |

**VFS** *Virtual File System*

**VM** *Virtual Machine*

# Table des figures

|     |                                                                                      |    |
|-----|--------------------------------------------------------------------------------------|----|
| 2.1 | Couches traversées par une application, de l'espace utilisateur au matériel. . . . . | 7  |
| 2.2 | Chemin d'un paquet entrant, de la carte réseau à l'application. . . .                | 7  |
| 2.3 | Descripteur d'anneau de réception, avec ses deux groupes de champs. .                | 8  |
| 2.4 | Trois formats d'en-tête, alignés sur la même grille de 16 bits. . . . .              | 8  |
| 3.1 | Occupation mémoire des trois allocateurs, à charge identique. . . . .                | 12 |
| 3.2 | Trace d'ordonnancement sur deux cœurs. Chaque bloc est un quantum. .                 | 14 |
| 4.1 | Débit soutenu, en milliers d'opérations par seconde. . . . .                         | 16 |
| 4.2 | Latence en fonction de la charge, pour deux configurations. . . . .                  | 17 |



# Liste des tableaux

|     |                                                                                          |    |
|-----|------------------------------------------------------------------------------------------|----|
| 2.1 | Sous-systèmes du noyau et unité de partage. . . . .                                      | 7  |
| 4.1 | Machines utilisées. Les deux sont bi-socket, donc NUMA. . . . .                          | 15 |
| 4.2 | Plan d'expérience : trois allocateurs × trois politiques × sept tailles. .               | 16 |
| 4.3 | Débit (kops/s) et latences par variante, écart relatif à la référence. .                 | 17 |
| 4.4 | Respect des cinq critères, par système. ✓ satisfait, ~ partiel, ✗ non satisfait. . . . . | 18 |
| A.1 | Symboles utilisés dans les modèles. . . . .                                              | 31 |



# Liste des listings

|     |                                                                     |    |
|-----|---------------------------------------------------------------------|----|
| 2.1 | Insertion d'une tâche dans la file d'exécution. . . . .             | 9  |
| A.1 | Options de compilation du noyau utilisées pour les mesures. . . . . | 31 |



# Détails d'implémentation

Cette annexe existe pour montrer que la numérotation bascule en lettres après #appendix().

## A.1 Paramètres de compilation

```
1 make defconfig
2 scripts/config --enable CONFIG_BPF_SYSCALL
3 scripts/config --enable CONFIG_NUMA_BALANCING
4 scripts/config --set-val CONFIG_HZ 1000
5 make -j"$(nproc)"
```

**List. A.1:** Options de compilation du noyau utilisées pour les mesures.

## A.2 Table des symboles

| Symbole   | Signification               |
|-----------|-----------------------------|
| $n$       | nombre de cœurs actifs      |
| $\lambda$ | taux d'arrivée des requêtes |
| $\mu$     | taux de service par cœur    |

**Tab. A.1:** Symboles utilisés dans les modèles.